

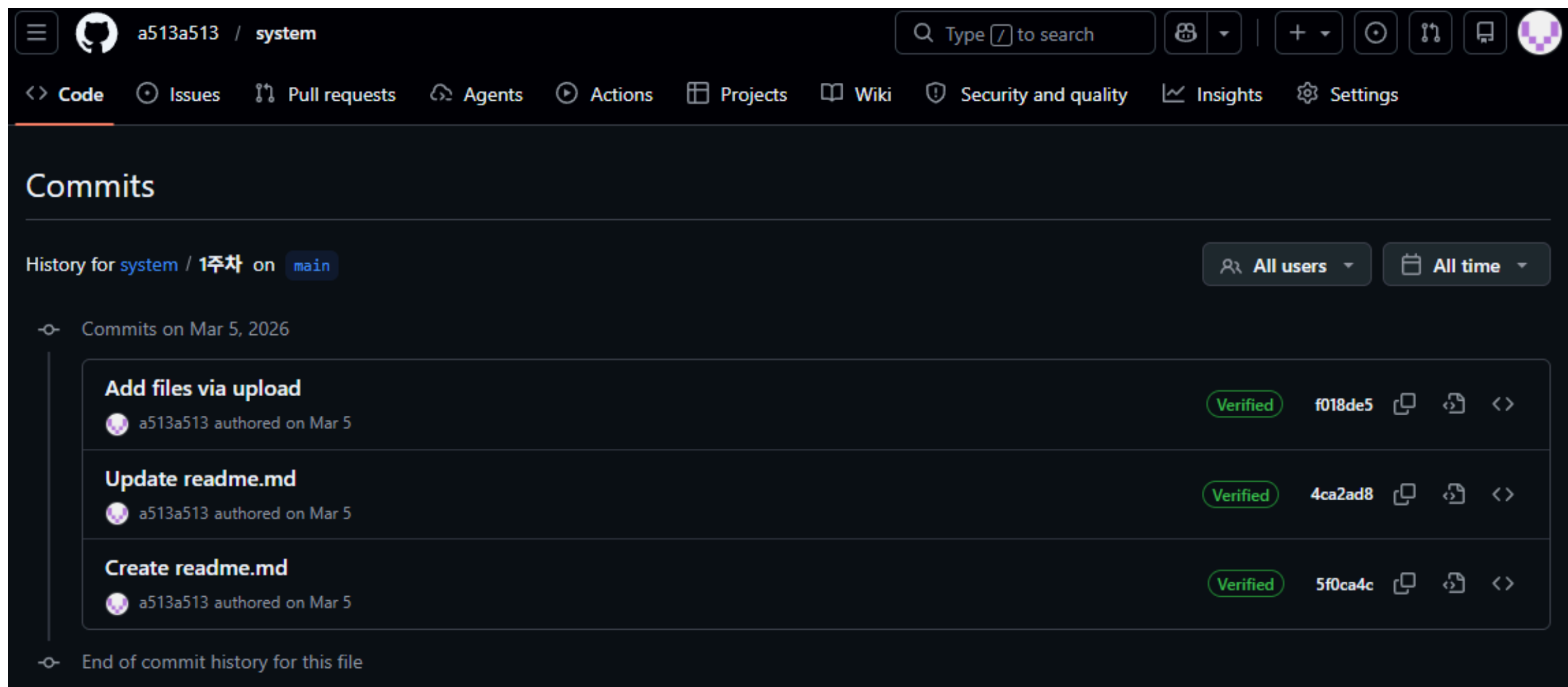
시스템 프로그래밍

깃허브 정리

<https://github.com/a513a513/system/tree/main>

2021963034 박준홍

1주차



2주차

a513a513 / system

<> Code

Issues

Pull requests

Agents

Actions

Projects

Commits

History for system / week2 on main

Commits on Mar 18, 2026

Create readme.md

a513a513 authored on Mar 18

End of commit history for this file

Linux 기초 및 명령어 학습 노트

📌 학습 피라미드 (Learning Pyramid)

학습 효과를 높이기 위한 개념으로, 배운 내용을 기억하는 비율을 나타냅니다.

- 수동적 학습: 강의 듣기(5%), 읽기(10%), 시청각 수업(20%), 시연 보기(30%)
- 능동적 학습: 토론(50%), 직접 실습해보기(75%), 다른 사람에게 가르치기(90%)

💡 Note: 이처럼 배운 명령어들을 깃허브에 직접 정리하고 누군가에게 설명하듯 기록하는 것은 학습 효율을 90%까지 끌어올리는 아주 좋은 방법입니다!

📖 도움말 및 매뉴얼 확인 명령어

명령어의 사용법이나 옵션이 기억나지 않을 때 사용하는 명령어들입니다.

- `man` [명령어] : Manual의 약자로, 명령어의 상세한 공식 매뉴얼 페이지를 엽니다. (종료: `q`)
- `info` [명령어] : `man` 페이지보다 좀 더 구조화되어 있고 상세한 하이퍼링크 형태의 정보를 제공합니다.
- `[명령어] --help` : 명령어의 핵심 사용법과 자주 쓰는 옵션들만 터미널 화면에 간단히 요약해서 출력합니다.

⚙️ 시스템 관리 및 패키지 설치 명령어

명령어 앞의 `#` 기호는 Root(최고 관리자) 권한으로 실행해야 함을 의미합니다.

- `# passwd root` : 시스템 최고 관리자인 `root` 계정의 비밀번호를 설정하거나 변경합니다.
- `# apt-get update` : 설치 가능한 소프트웨어 패키지들의 최신 목록을 서버로부터 가져와 업데이트합니다. (패키지 설치 전 필수 실행)
- `# apt install gcc` : C/C++ 프로그램 컴파일러인 `gcc` 패키지를 다운로드하고 설치합니다.

📁 디렉토리 이동 및 경로 기호

명령어 앞의 `$` 기호는 일반 사용자 권한으로 실행 중임을 의미합니다.

- `$ cd ~` : 현재 로그인한 사용자의 홈 디렉토리로 이동합니다. (`cd` 는 Change Directory의 약자)

[경로를 나타내는 특수 기호]

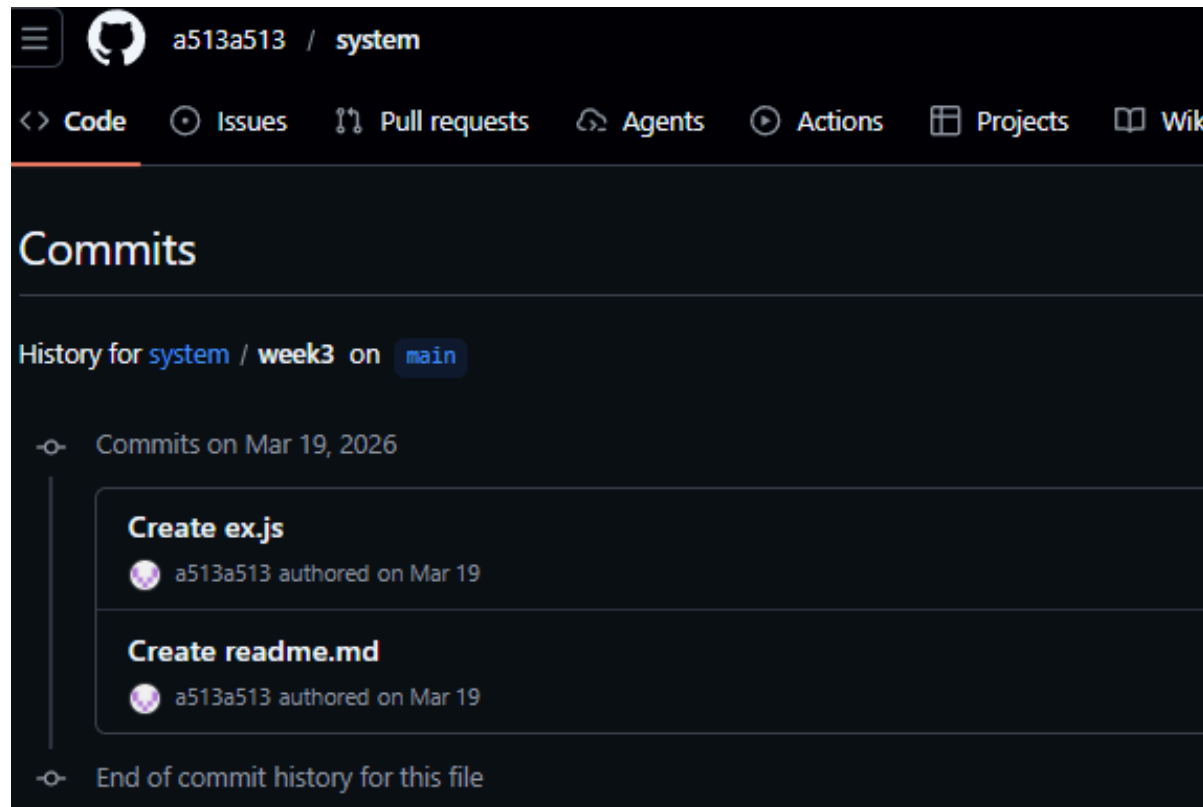
- `.` : 현재 디렉토리 (예: `./script.sh` -> 현재 폴더의 스크립트 실행)
- `..` : 부모 디렉토리 (현재 위치에서 한 단계 상위 폴더)
- `~` : 홈 디렉토리 (사용자의 기본 개인 작업 공간)
- `/` : 루트 디렉토리 (리눅스 파일 시스템의 최상위 경로)

🔍 파일 및 디렉토리 목록 확인 (ls 명령어)

현재 위치한 디렉토리 안의 내용물을 확인합니다.

- `ls` : 파일과 디렉토리의 이름만 간단히 목록으로 보여줍니다.
- `ls -l` : 파일의 권한, 소유자, 파일 크기, 수정 날짜 등 **상세 정보(long format)**를 함께 보여줍니다.
- `ls -al` : `ls -l` 의 결과에 더해, 이름이 `.` 으로 시작하는 **숨김 파일(all)**까지 모두 보여줍니다.
- `ll` : 대부분의 리눅스 배포판에서 `ls -al` 또는 `ls -l` 을 편하게 쓰기 위해 기본으로 설정해 둔 **단축 명령어**입니다.
- `alias` : 현재 시스템에 설정된 단축 명령어(별칭) 목록을 확인하거나, 새로운 단축 명령어를 직접 만들 때 사용합니다. (예: `alias ll='ls -al'`)

3주차



- 실행 오류 3가지 - syntex run time(exception) semantic

semantic을 제외한 나머지는 컴파일로 잡을 수 있지만, semantic은 디버깅으로만 잡을 수 있음 디버깅 - semantic을 잡으려고 디버깅 시도함

directory . - 디렉토리의 서브 폴더 .. - 디렉터리의 부모 폴더 / - root 디렉터리(모든 디렉토리의 출발점) ~ - home 디렉터리

path : 절대경로, 상대경로

```
mkdir(make directory) 0319 - 0319폴더를 생성한다.
```

\$ cat > test.txt 123456789 abcdefghi 컨트롤 + d = test.txt에 내용을 넣음(cat은 파일안에 내용을 넣음)

echo "내용입니다." >> file.txt = file.txt안에 "내용입니다."를 넣음

만약 파일이 없다면 >> 명령어를 넣어서 생성과 동시에 작성이 가능하고, > 는 파일을 생성하지 않지만 해당 지정 파일이 있다면 작성가능하고, 해당 파일 이름이 없는경우 삭제한다.

cat test.txt = test.txt안의 내용을 보여줌

\$ touch test2.txt = test2.txt에 아무것도 내용이 없는 파일을 만듦(만약 파일이 있는 거라면 타임스탬프가 해당 명령어를 쓴 시간으로 바뀜)

ll /bin = 기본 실행 명령어들의 나열

ll /usr/bin =

2.pdf 9page

파일은 디렉토리에 속하고, 사용자는 각각의 그룹에 속한다. 그룹의 설정에 따라 접근 가능하거나, 접근 불가 할 수 있다.

ll 열면 drwxr-xr-x 2 : drwxr-x--- 6 :

한 디렉토리를 삭제 하려고 하면 안의 폴더를 모두 삭제하여야 디렉토리를 지울 수 있음 rmdir "디렉토리이름"

하지만 rm -r "디렉토리이름" 명령어를 실행시키면 앞에 제약 없이 모두 지울 수 있음 ex) rm -r empDir/ : emDir의 서브 폴더를 모두 삭제 한다.

14주차 과제 ls /usr/bin 을 치면 명령어가 나오는데 30개 써서 c언어로 만들 라인 - \n 개수, 글자 수,

4주

  a513a513 / system

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#)

Commits

History for system / week4 on [main](#)

Commits on Mar 26, 2026

Add files via upload
 a513a513 authored on Mar 26

Create readme.md
 a513a513 authored on Mar 26

End of commit history for this file

.vimrc에 관련 코드를 넣으면 파일을 만들때 적용 그대로 환경 설정이 된다.
return 0은 정상적으로 종료 되었다는 보고 이고, return a는 정상적으로 종료되지 않았다는 보고 이다.

- .vimrc파일 내용 -

set number	" 줄 번호 표시
set relativenumber	" 상대 줄 번호 표시
set tabstop=4	" 탭 너비 4칸
set shiftwidth=4	" 자동 들여쓰기 너비 4칸
set expandtab	" 탭을 스페이스로 변환
set noautoindent	" 자동 들여쓰기 끄기
set nosmartindent	" 스마트 들여쓰기 끄기
syntax on	" 문법 강조

5주차

a513a513 / system

<> Code

Issues

Pull requests

Agents

Actions

Commits

History for system / week5 on main

Commits on Apr 8, 2026

Add files via upload

a513a513 authored on Apr 8

Add files via upload

a513a513 authored on Apr 8

Delete week5/upload

a513a513 authored on Apr 8

Create upload

a513a513 authored on Apr 8

Update readme.md

a513a513 authored on Apr 8

Create readme.md

a513a513 authored on Apr 8

End of commit history for this file

include와 lib 차이
헤더 파일(.h)이 들어가는 곳, 라이브러리 파일이 들어가는 곳

- 대표적인 버퍼 문제 -

```
#include
int main(){
    char string[20];
    char c;
    scanf("%s", string);
    scanf("%c", &c);
    printf("%s\n", string);
    printf("!!%c!!\n", c);
    return 0;
}
```

- scanf("%s", string); 에서 마지막 널문자가 버퍼에 남아 다음 scanf인 scanf("%c", &c)가 영향을 받는다.

- scanf 패턴 문제 -

```
#include
#include "student.h"

int main(int argc, char *argv[]){
    struct student rec;
    FILE *fp;
    if (argc != 2)
    {
        fprintf(stderr, "사용법: %s 파일이름\n", argv[0]);
        return 1;
    }
    fp = fopen(argv[1], "w");
    printf("%-9s %-7s %-4s\n", "학번", "이름", "점수");
    while (fscanf(fp, "%d %s %d", &rec.id, rec.name, &rec.score) == 3) ** // **
        printf("%10d %6s %6d\n", rec.id, rec.name, rec.score);
    fclose(fp);
    return 0;
}
```

** // ** 에서 scanf를 받은 횟수 만큼 숫자로 변환 되기에 "%d %s %d" 총 3번을 입력 받는다. 그렇기에 == 3이라는 조건이 만족이 되고 만약 사용자가 입력을 3개보다 적게 받거나, 3개 초과로 받는다면 그 자리에서 반복문을 빠져나와, return 0에 의해 종료 된다. scanf로 받으면 받은 횟수지만, printf를 정수로 반환하면 문자열의 개수만큼 반환이 된다.

ex)int = n;
n = printf("안녕하세요.")

n은 마지막 Null문자를 포함한 7로 반환되는걸 알 수 있다.

6주차

a513a513 / system

<> Code

Issues

Pull requests

Agents

Ad

Commits

History for system / week6 on main

Commits on Apr 9, 2026

Add files via upload

a513a513

 authored on Apr 9

End of commit history for this file

```
stdout이 1
stderr이 2

그다음으로 오는 파일이 3,4,5.. 이런식입니다.

ssize_t(data type) - 부호 있는 정수(signed)
size_t(data type) - 부호 없는 정수(0 ~ )
ex) - read() -

#include

ssize_t read ( int fd, void *buf, size_t nbytes );

파일 읽기에 성공하면 읽은 바이트 수, 파일 끝을 만나면 0,
실패하면 -1을 반환한다.

↳ 해당 코드는 읽은 바이트 수를 사용하기 때문에 1부터 다른 정수까지 return을 시킬 수 있다. 만약 실패하면 1로 반환한다 치면 중복 리턴이 올 수 있기 때
문에 음수에 해당하는 ssize_t를 사용하여 실패하면 -1로 반환하게 한다.

• fsz.c -

#include

#include

#include

#include

#define BUFSIZE 1024

/* 파일 크기를 계산 한다 */
int main(int argc, char *argv[])
{
    char buffer[BUFSIZE];
    int fd;
    ssize_t nread;
    long total = 0;

    if ((fd = open(argv[1], O_RDONLY)) == -1)
        perror(argv[1]);

    /* 파일의 끝에 도달할 때까지 반복해서 읽으면서 파일 크기 계산 */
    while( (nread = read(fd, buffer, BUFSIZE)) > 0 ) // nread는 파일을 읽었으면 1 반환
        total += nread; // 총 읽은 파일 개수

    close(fd);

    printf ("%s 파일 크기 : %ld 바이트 \n", argv[1], total);
    exit(0);
}

• write() -

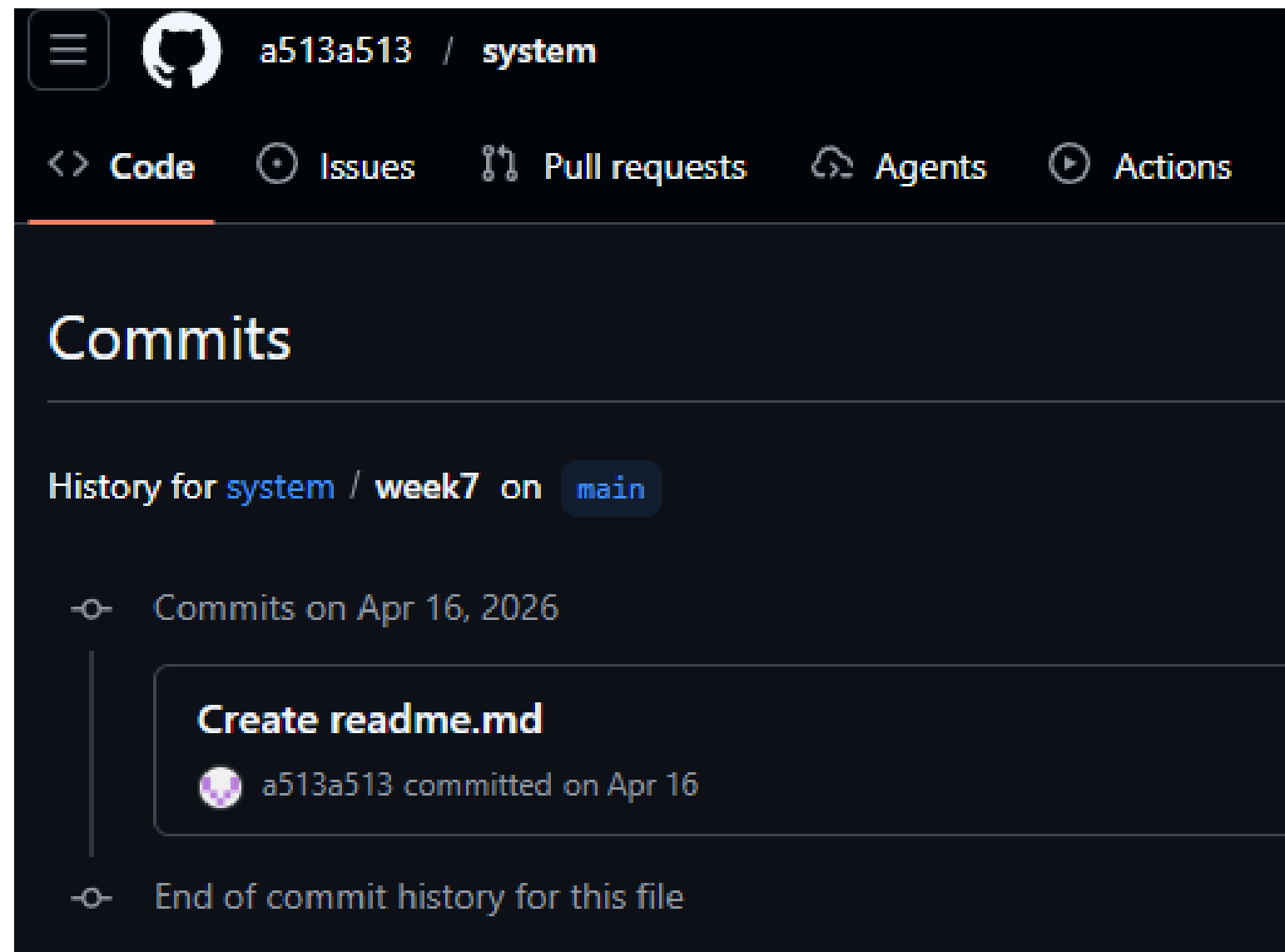
#include

ssize_t write (int fd, void *buf, size_t nbytes);

파일에 쓰기 성공하면 실제 쓰여진 바이트 수를 반환하고,
실패하면 -1을 반환한다.

↳ fd에 buf에 들어있는 nbytes값이 만큼 보냄. 하지만 쓰기에 실패한다면 -1을 반환함
```

7주차



readme.md

파일 시스템 구조: i-node와 데이터 블록

1. i-node (Index Node)

파일의 ****메타데이터(정보)****와 **데이터 위치**를 저장하는 핵심 자료구조입니다.

- 저장 정보: 파일 크기, 파일 소유자, 권한(rwx), 생성 및 수정 시간, 데이터 블록 위치, 링크 수
- 특징: 파일을 생성하면 무조건 **하나의 i-node**가 할당됩니다.

2. 데이터 블록 (Data Block)

실제 파일의 **내용**이 담겨 있는 저장 공간입니다.

- 특징: 내용이 없는 빈 파일이라면 데이터 블록이 할당되지 않을 수 있습니다.
- 할당 방식: 저장할 파일의 크기가 1개 블록의 크기를 넘어가면, 필요한 용량만큼 여러 개의 데이터 블록이 추가로 할당됩니다.

3. 파일 크기에 따른 블록 할당 구조

파일 시스템은 파일 크기에 커짐에 따라 **직접 블록** → **간접 블록** → **이중 간접 블록** 순으로 확장하여 데이터를 처리합니다.

블록 종류	연결 구조	활용	접근 속도
직접 블록	i-node 안에 데이터 블록 주소를 직접 저장	작은 파일 저장	가장 빠름
간접 블록	i-node가 블록 주소들을 저장한 블록을 가리킴	큰 파일 저장	느림
이중 간접 블록	i-node가 블록 주소를 저장한 블록을 가리키고, 그 블록이 또 다른 주소 블록을 가리킴	매우 큰 파일 저장	매우 느림

8주차

 a513a513 / system

[Code](#) [Issues](#) [Pull requests](#) [Agents](#) [Actions](#) [Projects](#) [Wiki](#)

Commits

History for [system](#) / **week8** on [main](#)

Commits on Apr 30, 2026

Add files via upload

 a513a513 authored on Apr 30

Create readme.md

 a513a513 authored on Apr 30

End of commit history for this file

readme.md

명령어 처리기 - Shell 다중 명령어 실행 (;) 프롬프트에서 ;를 써서 여러 명령어를 한 번에 실행할 수 있습니다.

Bash

date(서버 현재 시간)와 who(사용자 로그인 시간)를 함께 실행

date; who 파일 실행 권한 및 실행

Bash chmod +x test1 # test1 파일을 실행 가능하도록 만들 ./test1 # test1 파일 실행 변수 사용 () 변수를 참조할 때 앞에 붙여서 사용합니다.

Bash days=10 guest="Katie"
echo "\$guest checked in \$days days ago"

days=5
guest="Jessica" echo "\$guest checked in *daysdaysago*" 위치매개변수(숫자) 순서대로 들어오는 인자값을 의미합니다. (\$0 = argv[0], \$1 = argv[1]...)

[활용 예시: ./sc test4 실행 시]

sc 스크립트 (\$1 자리에 test4가 대입됨):

Bash vi \$1 chmod +x \$1 ./ \$1 test4 스크립트:

Bash value1=10 value2=\$value1 echo The resulting value is *value2* 종료상태확인(?) 직전 명령어의 return 값을 가져옵니다. 정상 종료(0)와 비정상 종료(0 제외 숫자)를 확인하여 분기 처리에 활용할 수 있습니다.

Bash return 5 echo \$? # 5 출력 명령어 치환 (백틱 `) 명령어를 에디터에서 실행할 때 쓰는 묶음입니다. 문자열이 아닌 명령어의 실행 결과로 치환됩니다.

Bash testing= `date` echo "The date and time are: " \$testing

9주

a513a513 / system

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Commits

History for system / week9 on main

Commits on Jun 4, 2026

Add files via upload

a513a513 authored 2 weeks ago

End of commit history for this file

readme.mdAdd files via upload2 weeks ago

readme.md

프로세스를 프로세스가 실행 할 수 있다
부모 프로세스는 자식 프로세스를 생성한 프로세스이며, 경우에 따라 백그라운드에서 실행되는 데몬 프로세스일 수도 있다.

명령어 다음에 & 기 들어가면 명령을 백그라운드에서 실행시킨다
ex) `$sleep 10 -> 10초동안 동작을 멈춤`
`$sleep 10 & -> 백그라운드에서 명령을 실행함`

bash의 자식 프로세스인 `sleep 10`, `$sleep 10 &` 의 명령이 있는데, 이 요청을 기다리냐 마느냐의 차이가 있다.
`sleep 10`명령은 자식 프로세스인 `sleep 10`명령을 부모가 기다려 아무 동작을 못하게 된다.
하지만 `$sleep 10 &` 명령은 자식 프로세스인 `$sleep 10 &` 명령을 부모프로세스가 기다리지 않고 바로 동작 할 수 있게 한다.

- 백그라운드를 전면 작업으로 바꿀 수 있음 -
프로세스는 각 작업번호가 있음(jobs로 확인 가능)
`$ fg %작업번호`
- 전면작업을 백그라운드로 바꿀 수도 있음 -
`$ bg %작업번호`
- 프로세스 강제 종료 -
`$ kill %작업번호`
- 프로세스 작업 끝나기를 기다림 -
`wait [PID]`로 해당 PID를 가진 프로세스가 작업이 종료가 되면 다음 프로세스를 실행 할 수 있음
ex) `$ (sleep 10; echo 1번 끝) &`
`[1] 1231`
`$ echo 2번 끝; wait 1231; echo 3번 끝`

실행 결과 : 2번끝 1번끝 3번끝
(백그라운드에서는 `echo`를 써도 출력이 화면에 안됨. 그렇기에 결과 값이 고정됨)

- `set-user-id(set user ID upon execution)` -
`root`의 권한을 사용자에게 권한을 주어 `root` 명령을 실행 가능하게 할 수 있게함
ex) `-rwsr-xr-x 1 root root 68208 Feb 6 2024 /usr/bin/passwd` <- `s`가 부여되어 있으면 루트 권한을 가져 실행 할 수 있음

* `set-user-id` 실행권한 설정 *

`$ chmod u+s 파일` OR `$ chmod 4755 파일` -> 파일에 `s`권한을 줌(개인)

* `set-group-id` 실행권한 설정 *

`$ chmod 2755 파일` OR `$ chmod g+s 파일` -> 파일에 `s`권한을 줌(그룹)

디렉토리를 생성하면 `i-node`의 번호와 다른 `i-node`) 그리고 접근권한 등을 가지게 됨

10주차

a513a513 / system

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Commits

History for system / week10 on main

Commits on Jun 4, 2026

Add files via upload

a513a513 authored 2 weeks ago

End of commit history for this file

Name	Last commit message	Last commit date
..		
upload	Add files via upload	2 weeks ago
readme.md	Add files via upload	2 weeks ago

readme.md

[07PDF]

- 컴퓨터 -
네트워크를 통과하는 *데이터 패킷(Data Packet)*의 목적지 주소를 추출하여 가장 적절한 경로를 설정하고, 그 경로에 따라 패킷을 다음 장치로 전송
데이터 패킷
큰 데이터를 나누어 보내는 일정량의 전송 단위

- 게이트웨이 -
게이트웨이는 서로 다른 네트워크(예: LAN과 WAN, 또는 서로 다른 프로토콜의 네트워크)를 연결해 데이터를 전달하는 장치 또는 기능

- IP -
네트워크에서 장치를 식별하는 숫자 주소입니다.

- TCP -
전송 프로토콜로 데이터 분할 (패킷으로 나눔), 순서 보장, 손실 시 재전송, 오류 검출을 하는 프로토콜입니다.

- Domain Name -
도메인 네임은 네트워크에서 사용하는 IP 주소를 사람이 이해하기 쉬운 문자 형태로 표현한 주소이며, DNS(Domain Name System) 서버를 통해 해당 도

- 여러 관련 명령어 -
\$ ip addr : 장치의 IP 주소 확인
\$ nslookup 호스트명(Domain name) : 호스트명에 대한 IP주소를 알려준다.
\$ finger 사용자명 : 사용자에 대한 자세한 정보를 알려준다.

[7PDF]

- 파일 잠금 -
한 프로세스가 그 영역을 읽거나 수정할 때 다른 프로세스의 접근을 제한, 만약 제한하지 않을 시 원치 않은 결과가 생길 가능성이 생긴다.

& 파일 잠금 관련 명령어
flock(<파일>, <커맨드>)
↳ LOCK_SH : 공유 잠금으로 한개 이상의 프로세스들이 파일에 대한 공유 잠금이 가능
↳ LOCK_EX : 배타 잠금으로 한번에 하나의 프로세스만이 파일에 대한 배타 잠금을 할 수 있고, 만약 파일에 이미 배타 잠금이 걸려
↳ LOCK_UN : 파일에 걸려 있는 잠금을 해제함
↳ LOCK_NB : 파일에 잠금이 걸려 있으면 기다리지 않고 반환함

- 레코드 잠금 -
파일의 일부 영역에 대해서 잠금한다.

& 레코드 잠금 관련 명령어
int fcntl(<파일>, <커맨드>, struct flock *lock)
↳ F_SETLK : 잠금 설정 ↳ 잠금 정보 구조체
↳ F_SETLKW : 잠금 설정
↳ F_GETLK : 잠금 정보 조회

11주차

a513a513 / system

<> Code

Issues

Pull requests

Agents

Commits

History for system / week11 on main

Commits on Jun 4, 2026

Add files via upload

a513a513 authored 2 weeks ago

End of commit history for this file

readme.md

Add files via upload

2 weeks

readme.md

[08PDF]

- 전면 처리 -

명령어를 입력하면 명령어가 전면에서 실행되며 명령어 실행이 끝날 때까지 쉼이 기다려 줌

- 후면 처리 -

명령어들을 후면에서 처리하고 전면에서는 다른 작업을 할 수 있으면 동시에 여러 작업을 수행할 수 있다.

사용법 : \$ 명령어 &

- 프로세스 -

실행중인 프로그램을 프로세스(process)라고 부르고, 각 프로세스는 유일한 프로세스 번호 PID를 갖습니다.

& 프로세스 관련 명령어

& ps : 현재 존재하는 프로세스들의 실행 상태를 요약해서 출력

& ps <옵션>

└-e : 시스템 전체 프로세스 출력

└-f : 자세한 정보

└-ef : 모든 프로세스 + 상세 정보

└-u 사용자 : 특정 사용자 프로세스만 출력

└aux : 전체 보기

& sleep <초> : 해당 시간만큼 프로세스 중지

& kill <프로세스 번호 OR %작업번호> : 해당 프로세스 번호 / 작업번호(프로세스 실행 순서)에 해당하는 프로세스를 강제 종료 시킴

& wait <프로세스번호> : 해당 프로세스 번호가 종료할때까지 기다림

& exit <종료코드> : 쉼을 종료하고 종료코드(exit code)을 부모 프로세스에 전달

[10PDF]

- Bash -

Linux/Unix 시스템에서 사용하는 명령어 해석기(shell) 이고, 사용자가 입력한 명령을 해석해서 os에 전달하는 역할을 합니다.

- 여러 명령어 -

\$ alias 이름=문자열 : 문자열의 명령을 <이름>이라는 별칭으로 축약해서 사용 가능함

\$ history [-r/h] [번호] : r(read), h(help)이고, 번호를 입력하면 최근에 사용한 명령을 번호값만큼 보여줌

\$!! : 바로 전 명령 실행

& !n : 이벤트 번호가 인 명령 재실행

! 시작스트링 : 시작스트링으로 시작하는 최근의 명령어를 실행

!? 서브스트링 : 서브스트링을 포함하는 최근의 명령어를 실행

강의자료 13pdf에서 37p 3문짜리로 발표(주제 : 부팅)

!c - 최근의 c로 시작한 명령을 시행

12주차

Commits

History for `system` / `week12` on `main`

Commits on Jun 17, 2026

- Update readme.md**
a513a513 authored now

Commits on Jun 4, 2026

- Add files via upload**
a513a513 authored 2 weeks ago
- Add files via upload**
a513a513 authored 2 weeks ago
- Create readme.md**
a513a513 authored 2 weeks ago

End of commit history for this file

a513a513 Update readme.md 09e1175 · now History

Preview Code Blame

- Fork - 현재 실행 중인 프로세스를 복제하여 새로운 자식 프로세스를 생성하는 시스템 호출입니다. 프로세스는 각 PID를 가집니다. 부모프로세스와 자식 프로세스를 구분하기 위해 PID반환값을 달리하는데 부모프로세스는 정수값을 가지고, 자식 프로세스는 PID반환값을 0으로 가집니다. 이를 활용하여 PID가 0이면 자식 프로세스에게 명령을 내리도록 할 수 있고, 혹은 0이 아니면 부모 프로세스에게 명령을 내리도록 설계를 할 수 있습니다. 하지만 PID반환값이 0이라고 해서 자식 프로세스의 PID가 0은 아닙니다. 모든 프로세스는 운영체제가 부여하는 양수 PID를 가집니다. 자식 프로세스도 한 프로세스이기에 PID값을 가집니다. 다만 PID값이랑 PID반환값이랑 구별해야하는 점은 PID반환값은 부모/자식을 구분하기 위해 `fork()`가 돌려주는 값이며, PID값은 그 프로세스의 고유 번호라는 것입니다. `fork()`를 사용하면 부모 프로세스와 자식 프로세스가 생성되며, 어느 프로세스가 먼저 실행될지는 운영체제의 스케줄러가 결정합니다. 부모 프로세스가 자식 프로세스의 종료를 기다리게 하려면 `wait()` 함수를 사용하는 방법도 있습니다.
- `exec` - `exec()`는 현재 프로세스가 실행하는 프로그램만 바꾸는 함수입니다. 프로세스에 부여된 PID는 그대로 유지되며, 기존에 실행 중이던 프로그램이 새로운 프로그램으로 덮어써져 실행되게 되는 것 입니다.

명령어

[🔗 1. 파일 및 디렉토리 관리 명령어](#)

이 명령어들은 파일 시스템을 조작하고 관리하는 데 기본적으로 사용됩니다.

1. pwd

- 기능: 현재 작업 디렉토리의 전체 경로를 출력합니다.

```
#include <stdio.h>
#include <direct.h> // 윈도우 디렉토리 관련 헤더

int main() {
    char buf[1024];
    // _getcwd: 현재 작업 디렉토리의 절대 경로를 버퍼에 복사
    if (_getcwd(buf, sizeof(buf))) printf("%s\n", buf);
    return 0;
}
```

2. ls

- 기능: 디렉토리의 내용(파일 및 하위 디렉토리)을 나열합니다.

```
#include <stdio.h>
#include <io.h> // 윈도우 파일 탐색 관련 헤더

int main() {
    struct _finddata_t f; // 파일 정보를 저장할 구조체
    // 현재 디렉토리의 모든 파일("*.*)을 검색 시작
    intptr_t h = _findfirst("*.*", &f);
    if (h != -1L) {
        do {
            // 숨김 파일(.)로 시작하는 것은 제외하고 출력
            if (f.name[0] != '.') printf("%s\n", f.name);
        } while (_findnext(h, &f) == 0); // 다음 파일 찾기
        _findclose(h); // 핸들 닫기
    }
    return 0;
}
```

3. mkdir

- 기능: 새로운 디렉토리를 생성합니다.

```
#include <direct.h>

int main(int argc, char *argv[]) {
    // 인자가 전달되었다면 _mkdir 함수로 디렉토리 생성
    if (argc > 1) _mkdir(argv[1]);
    return 0;
}
```

4. rmdir

- 기능: 비어 있는 디렉토리를 삭제합니다.

```
#include <direct.h>

int main(int argc, char *argv[]) {
    // 인자가 전달되었다면 _rmdir 함수로 빈 디렉토리 삭제
    if (argc > 1) _rmdir(argv[1]);
    return 0;
}
```

5. touch

- 기능: 빈 파일을 생성하거나 파일의 수정 시간을 갱신합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        // "a" (append) 모드로 열어 파일이 없으면 생성, 있으면 내용을 유지함
        FILE *f = fopen(argv[1], "a");
        if (f) fclose(f); // 열린 파일 닫기
    }
    return 0;
}
```

6. rm

- 기능: 파일이나 디렉토리를 삭제합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    // remove: 표준 c 라이브러리의 파일 삭제 함수
    if (argc > 1) remove(argv[1]);
    return 0;
}
```

7. rm -f

- 기능: 강제로 파일을 삭제합니다. (에러 무시)

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    // 인자가 3개(ex: rm.exe -f test.txt)일 때 3번째 인자(파일)를 삭제
    if (argc > 2) remove(argv[2]);
    return 0;
}
```

8. basename

- 기능: 경로에서 디렉토리를 제외한 파일명만 추출합니다.

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        // 경로에서 마지막 백슬래시(\)나 슬래시(/)의 위치를 찾을
        char *p = strrchr(argv[1], '\\');
        if (!p) p = strrchr(argv[1], '/');
        // 슬래시가 있으면 그 다음 글자부터 출력, 없으면 원본 그대로 출력
        printf("%s\n", p ? p + 1 : argv[1]);
    }
    return 0;
}
```

9. dirname

- 기능: 파일 경로에서 디렉토리 경로만 추출합니다.

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        // 마지막 구분자(\ 또는 /)를 찾을
        char *p = strrchr(argv[1], '\\');
        if (!p) p = strrchr(argv[1], '/');
        if (p) {
            *p = '\\0'; // 마지막 구분자를 널 문자(\0)로 바꿔 문자열을 자름
            printf("%s\n", argv[1]);
        }
        else printf(".\n"); // 경로 구분자가 없으면 현재 경로(.) 출력
    }
    return 0;
}
```

10. file

- 기능: 파일 또는 디렉토리 형식을 판별합니다.

```
#include <stdio.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        struct _stat s;
        if (_stat(argv[1], &s) == 0) {
            // st_mode 비트 연산으로 파일인지 디렉토리(_S_IFDIR)인지 판별
            if (s.st_mode & _S_IFDIR) printf("%s: directory\n", argv[1]);
            else printf("%s: regular file\n", argv[1]);
        }
    }
    return 0;
}
```

II. 텍스트 및 파일 출력 명령어

파일 내용을 터미널에 출력하거나 텍스트를 처리하는 데 사용됩니다.

11. echo

- 기능: 입력받은 문자열을 터미널에 그대로 출력합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    // argv[1]부터 마지막 인자까지 공백을 두고 출력
    for (int i = 1; i < argc; i++) printf("%s ", argv[i]);
    printf("\n"); // 마지막에 줄바꿈
    return 0;
}
```

12. cat

- 기능: 파일의 내용을 터미널에 출력합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *f = fopen(argv[1], "r"); // "r" (읽기) 모드로 파일 열기
        int c;
        if (f) {
            // 파일의 끝(EOF)에 도달할 때까지 한 글자씩 읽어서 화면에 출력
            while ((c = fgetc(f)) != EOF) putchar(c);
            fclose(f);
        }
    }
    return 0;
}
```

13. cat -n

- 기능: 파일의 내용을 줄 번호와 함께 출력합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 2) {
        FILE *f = fopen(argv[2], "r");
        char line[1024];
        int n = 1; // 줄 번호 저장 변수
        if (f) {
            // fgets로 한 줄씩 읽은 뒤, 줄 번호(%d)와 함께 출력
            while (fgets(line, sizeof(line), f)) printf("%d %s", n++, line);
            fclose(f);
        }
    }
    return 0;
}
```

14. head

- 기능: 파일의 시작 부분(기본 10줄)을 출력합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *f = fopen(argv[1], "r");
        char line[1024];
        int count = 0; // 출력된 줄 수 추적
        if (f) {
            // 읽은 줄이 10줄 미만일 때까지만 루프 실행
            while (count < 10 && fgets(line, sizeof(line), f)) {
                printf("%s", line);
                count++;
            }
            fclose(f);
        }
    }
    return 0;
}
```

15. grep

- 기능: 파일 내용에서 특정 패턴(문자열)을 검색합니다.

```
#include <stdio.h>
#include <string.h>

int main(int argc, char *argv[]) {
    if (argc > 2) {
        FILE *f = fopen(argv[2], "r");
        char line[1024];
        if (f) {
            while (fgets(line, sizeof(line), f)) {
                // strstr: 읽어들인 줄(line) 안에 검색어(argv[1])가 포함되어 있는지 확인
                if (strstr(line, argv[1])) printf("%s", line);
            }
            fclose(f);
        }
    }
    return 0;
}
```

16. wc

- 기능: 파일의 줄 수, 단어 수, 문자 수를 계산하여 출력합니다.

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        FILE *f = fopen(argv[1], "r");
        int c, nl = 0, nw = 0, nc = 0, inw = 0; // 줄, 단어, 글자, 단어 내부여부 플래그
        if (f) {
            while ((c = fgetc(f)) != EOF) {
                nc++; // 글자 수 증가
                if (c == '\n') nl++; // 줄바꿈 문자면 줄 수 증가
                // 공백, 줄바꿈, 탭이면 단어 내부 플래그 해제
                if (c == ' ' || c == '\n' || c == '\t') inw = 0;
                else if (!inw) { inw = 1; nw++; } // 새 단어 진입 시 단어 수 증가
            }
            printf("%d %d %d %s\n", nl, nw, nc, argv[1]);
            fclose(f);
        }
    }
    return 0;
}
```


III. 시스템 정보 및 네트워크 명령어

시스템의 상태를 확인하고 네트워크 연결을 테스트하는 데 사용됩니다.

17. whoami

- 기능: 현재 로그인한 사용자의 이름을 출력합니다.

```
#include <stdio.h>
#include <windows.h> // 윈도우 API 헤더

int main() {
    char buf[256];
    DWORD len = sizeof(buf);
    // GetUserName: 윈도우 시스템에 로그인된 현재 사용자 이름을 가져옴
    if (GetUserName(buf, &len)) printf("%s\n", buf);
    return 0;
}
```

18. hostname

- 기능: 시스템의 호스트(컴퓨터) 이름을 표시합니다.

```
#include <stdio.h>
#include <windows.h>

int main() {
    char buf[256];
    DWORD len = sizeof(buf);
    // GetComputerName: 윈도우 시스템의 컴퓨터 이름을 가져옴
    if (GetComputerName(buf, &len)) printf("%s\n", buf);
    return 0;
}
```

19. uname

- 기능: 시스템 운영체제 정보를 출력합니다.

```
#include <stdio.h>

int main() {
    // 윈도우 환경에서 빌드 및 실행되므로 정적으로 출력
    printf("Windows\n");
    return 0;
}
```

20. date

- 기능: 현재 날짜와 시간 정보를 출력합니다.

```
#include <stdio.h>
#include <time.h> // 시간 관련 헤더

int main() {
    time_t t = time(NULL); // 현재 시간의 타임스탬프 획득
    // ctime: 타임스탬프를 사람이 읽기 쉬운 문자열 형태로 변환하여 출력
    printf("%s", ctime(&t));
    return 0;
}
```

21. uptime

- 기능: 시스템 부팅 후 경과된 시간을 보여줍니다.

```
#include <stdio.h>
#include <windows.h>

int main() {
    // GetTickCount: 시스템 부팅 후 경과된 시간을 밀리초(ms) 단위로 반환
    DWORD ticks = GetTickCount();
    // 시간, 분, 초 단위로 변환 계산
    int s = (ticks / 1000) % 60, m = (ticks / 60000) % 60, h = (ticks / 3600000);
    printf("Uptime: %d hours, %d minutes, %d seconds\n", h, m, s);
    return 0;
}
```

22. stat

- 기능: 파일의 상세 상태 정보(크기 등)를 출력합니다.

```
#include <stdio.h>
#include <sys/stat.h> // 파일 상태 정보 헤더

int main(int argc, char *argv[]) {
    if (argc > 1) {
        struct _stat s;
        // _stat 함수로 파일의 정보를 가져와 구조체에 저장
        if (_stat(argv[1], &s) == 0) printf("Size: %ld bytes\n", s.st_size);
    }
    return 0;
}
```

23. printenv

- 기능: 시스템의 모든 환경 변수를 출력합니다.

```
#include <stdio.h>

// environ: 운영체제의 모든 환경변수를 담고 있는 전역 포인터 배열
extern char **environ;

int main() {
    // 배열의 끝(NULL)을 만날 때까지 모든 환경변수를 출력
    for (int i = 0; environ[i] != NULL; i++) printf("%s\n", environ[i]);
    return 0;
}
```

24. env

- 기능: 환경 변수를 출력합니다. (printenv 와 동일하게 구현)

```
#include <stdio.h>

extern char **environ;

int main() {
    for (int i = 0; environ[i] != NULL; i++) printf("%s\n", environ[i]);
    return 0;
}
```

25. ping

- 기능: 네트워크 호스트와의 연결성을 테스트합니다.

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        char cmd[256];
        // 윈도우 시스템의 기본 ping 명령어를 문자열로 조합하여 실행
        sprintf(cmd, "ping %s", argv[1]);
        system(cmd);
    }
    return 0;
}
```

26. ping -c

- 기능: 지정된 횟수만큼 패킷을 보내 연결성을 테스트합니다.

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 3) {
        char cmd[256];
        // 리눅스의 '-c' 옵션은 윈도우 ping 명령어의 '-n' (전송 횟수)과 같음
        sprintf(cmd, "ping -n %s %s", argv[2], argv[3]);
        system(cmd);
    }
    return 0;
}
```

27. curl

- 기능: URL을 통해 웹 데이터를 다운로드하거나 요청합니다.

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        char cmd[256];
        // 최근 윈도우 10/11에는 curl이 내장되어 있어 시스템 명령으로 호출 가능
        sprintf(cmd, "curl %s", argv[1]);
        system(cmd);
    }
    return 0;
}
```

IV. 프로세스 및 시스템 제어 명령어

터미널을 제어하거나 프로세스의 실행 흐름을 관리합니다.

28. clear

- 기능: 터미널 화면을 깨끗하게 지웁니다.

```
#include <stdlib.h>

int main() {
    // 윈도우 cmd의 화면 지우기 명령어인 'cls'를 시스템 함수로 실행
    system("cls");
    return 0;
}
```

29. sleep

- 기능: 프로세스를 지정된 시간 동안 일시 중단합니다.

```
#include <stdlib.h>
#include <windows.h>

int main(int argc, char *argv[]) {
    if (argc > 1) {
        // Sleep 함수는 밀리초 단위이므로 초 단위 인자에 1000을 곱함
        Sleep(atoi(argv[1]) * 1000);
    }
    return 0;
}
```

30. exit

- 기능: 프로그램을 정상적으로 종료합니다.

```
#include <stdlib.h> // exit 함수가 포함된 헤더

int main() {
    // exit(0): 프로그램 정상 종료 (상태 코드 0 반환)
    exit(0);
    return 0;
}
```

깃허브 : 12점

3주 가량 늦게 올린게 존재

명령어 : 15점

30개 다 작성하여 깃허브에 업로드함

총점 : 27점